

Antigravity 2.0 — Full-Stack App Prompt

Stack: Next.js 15 (App Router) · TypeScript · Tailwind CSS · Supabase · Vercel

1. Project Directives (AGENT.md)

Place this file at the root of your project as `AGENT.md`. Antigravity reads it on every session to maintain project memory and enforce structural consistency.

markdown

Directives

Folder Structure

- /app → Next.js App Router pages & layouts
- /app/api → API Route Handlers (server-side)
- /components → Shared React components
- /components/ui → Primitive UI components (Button, Input, Modal, etc.)
- /lib → Utility functions, helpers, constants
- /lib/supabase → Supabase client initializers (browser + server)
- /hooks → Custom React hooks
- /types → Global TypeScript interfaces & enums
- /styles → Global CSS, Tailwind config overrides
- /public → Static assets
- /tests → Unit & integration tests (Vitest)
- /supabase → DB migrations, seed files, RLS policies

Naming Conventions

- Components: PascalCase (UserProfile.tsx)
- Hooks: camelCase prefixed with "use" (useSession.ts)
- API routes: kebab-case route segments (/app/api/auth/callback/route.ts)
- DB tables: snake_case (user_profiles, project_items)
- ENV vars: SCREAMING_SNAKE prefixed with NEXT_PUBLIC_ for client-safe vars

Orchestration

- Always generate TypeScript – no plain JS files
- Always add JSDoc comments to exported functions
- Always add Zod validation schemas for all API inputs
- Use Server Components by default; add "use client" only when needed
- Use server-side Supabase client in Route Handlers and Server Components
- Use browser-side Supabase client only in Client Components

Execution Commands

- dev: npm run dev
- build: npm run build
- test: npm run test
- lint: npm run lint
- db:push: npx supabase db push
- db:migrate: npx supabase migration new <name>

2. Full Prompt to Paste into Antigravity 2.0

Build a full-stack web application using the following stack and architecture. Follow every instruction precisely and generate all files needed for a production-ready deployment.

TECH STACK

Frontend:

- Framework: Next.js 15 with App Router (TypeScript)
- Styling: Tailwind CSS v4
- UI Library: shadcn/ui (initialized with `npx shadcn@latest init`)
- Form Handling: React Hook Form + Zod for validation
- State: Zustand for global client state
- Data Fetching: TanStack Query v5 for client-side server state
- Icons: Lucide React

Backend:

- API Layer: Next.js Route Handlers (/app/api/**/*.ts)
- Auth: Supabase Auth (email/password + OAuth with Google)
- Database: Supabase (PostgreSQL via @supabase/ssr + @supabase/supabase-js)
- Storage: Supabase Storage (for file/image uploads if required)
- Validation: Zod on all API inputs

Infrastructure:

- Hosting: Vercel (via vercel.json config + environment variables)
- Database: Supabase (cloud-hosted PostgreSQL)
- CI/CD: Vercel Git integration (auto-deploy on push to main)

APPLICATION – [REPLACE THIS SECTION]

[Replace the block below with your app description. Be specific.]

App Name: [YOUR APP NAME]

Description: [Describe what your app does in 2-3 sentences]

Target Users: [Who is this for?]

Core Features:

1. [Feature 1]
2. [Feature 2]
3. [Feature 3]
4. [Feature 4]

Example (replace with yours):

App Name: TaskFlow

Description: A project management tool where teams can create
workspaces, assign tasks, and track progress in real time.

Target Users: Small to mid-size development teams.

Core Features:

1. User auth with email/password and Google OAuth
2. Create workspaces; invite team members by email
3. Kanban board with drag-and-drop task management
4. Real-time updates using Supabase Realtime subscriptions

SUPABASE SETUP

1. Install packages:

```
npm install @supabase/supabase-js @supabase/ssr
```

2. Create `/lib/supabase/client.ts` (browser):

```
import { createBrowserClient } from '@supabase/ssr'
export const createClient = () =>
  createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
```

3. Create `/lib/supabase/server.ts` (server – cookies):

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
export const createClient = async () => {
  const cookieStore = await cookies()
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    { cookies: { getAll: () => cookieStore.getAll(),
      setAll: (cs) => cs.forEach(({ name, value, options }) =>
        cookieStore.set(name, value, options)) } }
  )
}
```

4. Create `/app/api/auth/callback/route.ts`:

Handle Supabase Auth email confirmation + OAuth redirects.

Exchange the ``code`` query param for a session using

``supabase.auth.exchangeCodeForSession(code)``.

Redirect to `/dashboard` on success, `/auth/error` on failure.

5. Create Next.js middleware at `/middleware.ts`:

- Refresh expired Auth tokens on every request
- Protect all `/dashboard/**` routes (redirect to `/login` if no session)
- Allow public access to `/`, `/login`, `/signup`, `/api/auth/callback`

6. Database Schema (generate SQL migrations in `/supabase/migrations/`):

- Enable Row Level Security (RLS) on all tables

- users table: extends Supabase auth.users via trigger
- [Add your domain-specific tables based on app features]
- Every table must have: id (uuid, default gen_random_uuid()), created_at (timestampz, default now()), updated_at (timestampz)
- Add a trigger function `handle\_updated\_at()` for updated_at columns
- RLS policies: authenticated users can only read/write their own rows (use `auth.uid() = user\_id` pattern)

VERCEL DEPLOYMENT CONFIG

1. Create /vercel.json:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": ".next",
  "framework": "nextjs",
  "regions": ["sin1"],
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Frame-Options",          "value": "DENY" },
        { "key": "X-Content-Type-Options",  "value": "nosniff" },
        { "key": "Referrer-Policy",         "value": "strict-origin-when-
cross-origin" },
        { "key": "Permissions-Policy",      "value": "camera=(), microphone=
(), geolocation=()" }
      ]
    }
  ]
}
```

2. Required Environment Variables (set in Vercel Project Settings):

NEXT_PUBLIC_SUPABASE_URL	→ your Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	→ your Supabase anon/public key
SUPABASE_SERVICE_ROLE_KEY	→ your Supabase service role key (server-only, never expose to client)
NEXT_PUBLIC_SITE_URL	→ https://your-vercel-domain.vercel.app

3. In Supabase Dashboard → Authentication → URL Configuration:

Site URL:	https://your-vercel-domain.vercel.app
Redirect URLs:	https://your-vercel-domain.vercel.app/api/auth/callback

Also add localhost: http://localhost:3000/api/auth/callback (for local dev)

FILE STRUCTURE TO GENERATE

Generate ALL of the following files:

```
/
├── AGENT.md                ← Agent directives (see Section 1)
├── next.config.ts          ← Next.js config (images, env, etc.)
├── tailwind.config.ts      ← Tailwind v4 config
├── tsconfig.json           ← Strict TypeScript config
├── vercel.json             ← Vercel deployment config
├── .env.local.example      ← Template for env vars (no real values)
├── middleware.ts           ← Auth session refresh + route protection
|
├── app/
|   ├── layout.tsx          ← Root layout with Providers
|   ├── page.tsx            ← Landing/marketing page
|   ├── (auth)/
|   |   ├── login/page.tsx
|   |   └── signup/page.tsx
|   ├── (protected)/
|   |   └── dashboard/page.tsx ← Main authenticated view
|   └── api/
|       └── auth/callback/route.ts ← Supabase OAuth/email callback
|
├── components/
|   ├── ui/                 ← shadcn/ui primitives
|   ├── auth/
|   |   ├── LoginForm.tsx
|   |   └── SignupForm.tsx
|   └── [domain components based on features]
|
├── lib/
|   ├── supabase/
|   |   ├── client.ts       ← Browser Supabase client
|   |   └── server.ts        ← Server Supabase client (cookies)
|   ├── validations/        ← Zod schemas
|   └── utils.ts             ← cn() helper + misc utils
|
├── hooks/
|   └── useUser.ts           ← Auth session hook
|
├── types/
|   └── database.types.ts    ← Generated Supabase type definitions
|
└── supabase/
    └── migrations/
        └── 0001_init.sql    ← Initial DB schema + RLS policies
```

CODE QUALITY REQUIREMENTS

- All components must be typed with explicit TypeScript interfaces (no ``any``)
- All API route handlers must validate input with Zod and return typed responses
- All database queries must handle errors explicitly (no unhandled promise rejections)
- Use ``next/image`` for all images (never raw `<img>` tags)
- Use ``next/link`` for all internal navigation
- All forms: loading states, error messages, and success feedback
- Responsive design: mobile-first Tailwind breakpoints (sm, md, lg)
- Accessibility: semantic HTML, ARIA labels on interactive elements, keyboard navigation on modals and dropdowns

LOCAL DEVELOPMENT SETUP SCRIPT

After generating all files, generate a setup.sh that:

1. Runs ``npm install``
2. Runs ``npx shadcn@latest init --yes --defaults``
3. Copies ``.env.local.example`` to ``.env.local``
4. Prints instructions to fill in env vars from Supabase dashboard
5. Runs ``npm run dev``

3. MCP Servers to Connect in Antigravity 2.0

In the Antigravity IDE, connect these MCP servers for live backend integration:

MCP Server	Purpose
@supabase/mcp-server-supabase	Run DB migrations, query tables, manage RLS policies live
@vercel/mcp-server	Trigger deploys, inspect logs, manage env vars
@modelcontextprotocol/server-fileSystem	Read/write local project files
@upstash/mcp-server	(Optional) Redis caching layer

Install the Supabase MCP server:

```
bash
```

```
npx @supabase/mcp-server-supabase --access-token YOUR_SUPABASE_PAT
```

4. Supabase Type Generation

After your schema is set, regenerate TypeScript types with:

```
bash
```

```
npx supabase gen types typescript \  
  --project-id YOUR_PROJECT_REF \  
  --schema public \  
  > types/database.types.ts
```

Add this as an npm script in `package.json`:

```
json
```

```
"db:types": "supabase gen types typescript --project-id YOUR_PROJECT_REF --schema pu
```

5. Vercel Deployment Checklist

- ☐ Push code to GitHub (or GitLab/Bitbucket)
- ☐ Connect repo to Vercel via vercel.com/new
- ☐ Set all environment variables in Vercel Project Settings → Environment Variables
- ☐ Add your Vercel domain to Supabase Auth Redirect URLs
- ☐ Set `NEXT_PUBLIC_SITE_URL` to your production Vercel URL
- ☐ Trigger first deploy and verify `/api/auth/callback` returns 200
- ☐ Enable Vercel Analytics (optional, free tier)
- ☐ Enable Vercel Speed Insights (optional, free tier)